UNIT 5

Q1) Describe different methods of record blocking with the help of a neat diagram.

Record blocking is a file storage technique that organizes multiple logical records into physical blocks on storage devices (like hard disks or SSDs). Since I/O devices operate in terms of blocks rather than individual records, grouping records into blocks reduces the overhead of accessing storage and maximizes efficiency.

Why Use Record Blocking?

Efficiency: Reading and writing multiple records in one I/O operation is faster than handling each record separately.

Space Utilization: Blocks minimize storage wastage by aligning logical records to the physical block structure of the device.

Reduced Overhead: Fewer I/O operations lead to reduced CPU overhead and faster data processing.

Methods of Record Blocking

1. Fixed Blocking

Definition: Logical records of fixed size are packed into physical blocks. Each block contains an exact, integer number of records.

Characteristics:

All records have the same size.

Block size is a multiple of the record size.

No internal fragmentation occurs if the block size perfectly matches the record size.

Advantages:

Simple implementation and efficient processing.

No space is wasted if records fit exactly into blocks.

Disadvantages:

Requires records to have uniform size.

Not suitable for systems with variable-sized records.

Example:

Block size = 400 bytes, Record size = 100 bytes.

Each block will store exactly 4 records.

2. Variable Blocking (Spanned Blocking)

Definition: Logical records of variable size are packed into blocks, and records may span across multiple blocks if a record cannot fit entirely in one block.

Characteristics:

Allows records of varying sizes.

Uses pointers or continuation markers to indicate the start or continuation of a record in another block.

Advantages:

Optimizes space by accommodating variable-sized records.

No internal fragmentation as records can span across blocks.

Disadvantages:

Complex implementation due to the need for markers or pointers.

Retrieval can be slower as records may span multiple blocks.

Example:

Record 1 = 250 bytes, Record 2 = 600 bytes.

If block size = 500 bytes:

Block 1 contains Record 1 and part of Record 2 (500 bytes in total).

Block 2 contains the remainder of Record 2 and possibly other records.

3. Variable Blocking (Unspanned Blocking)

Definition: Logical records of variable size are stored entirely within a block, and no record spans multiple blocks. If there is insufficient space in a block for a record, the remaining space in the block is padded, and the record is moved to the next block.

Characteristics:

Records are stored completely within a single block.
Padding is added to the unused portion of the block, leading to internal fragmentation.
Advantages:
Simplifies record retrieval since records are contained within a single block.
Useful for systems where record integrity must be preserved.
Disadvantages:
Wastes space due to padding (internal fragmentation).
Less efficient space utilization compared to spanned blocking.
Example:
Block size = 500 bytes, Record 1 = 300 bytes, Record 2 = 250 bytes.
Block 1 stores Record 1 and leaves 200 bytes of padding.
Record 2 is moved to Block 2.

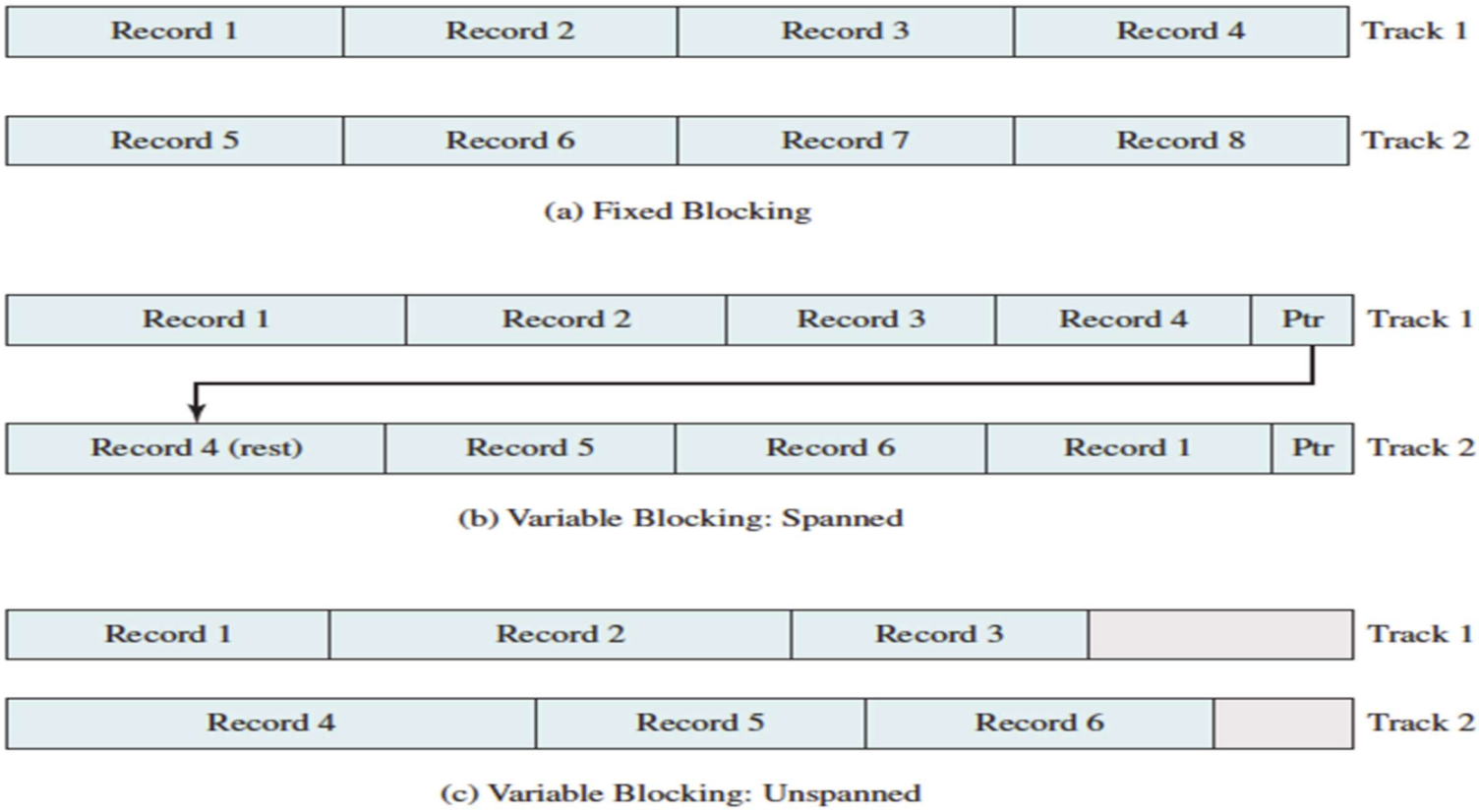


Figure 12.8 Record-Blocking Methods

Q2) Functions of a File Management System (FMS)
A **File Management System (FMS)** is a component of the operating system responsible for managing and organizing files on storage devices. It provides the interface between the user and the physical storage, ensuring that data is stored, retrieved, and manipulated efficiently.
1. File Creation and Deletion
Creation: The FMS allows users to create files with specified names, attributes, and data. It assigns storage space for the file and updates the directory structure
Deletion: The FMS facilitates deleting files, releasing their occupied storage space, and updating directory structures.
Example: Creating a .txt file in a folder or deleting an outdated document.

2. File Organization and Storage
The FMS organizes files into logical structures like directories and folders to enable easy access.
It determines how files are stored on the physical disk (e.g., contiguous allocation, linked allocation, or indexed allocation).
Example: Storing files in folders like Documents, Pictures, etc., based on the user's preference.

3. File Access and Manipulation
Provides methods to read, write, and modify files:
- **Read**: Retrieve data from a file.
- **Write**: Add new data to a file.
- **Update**: Modify existing data in a file.

Implements access methods like sequential access, direct access, or indexed access.
Example: Opening a .docx file to edit or reading a .csv file for analysis.

4. Security and Access Control
Ensures that files are accessed only by authorized users.
Implements file permissions (e.g., read, write, execute) to regulate user actions.
Provides encryption mechanisms to secure sensitive data.
Example: A file marked as "read-only" cannot be edited by unauthorized users.

5. Directory Management
Maintains the hierarchical organization of files in directories or folders.
Keeps metadata for each file, such as name, size, creation date, last modification date, and permissions.
Provides operations like creating, renaming, or deleting directories.
Example: Navigating a folder structure like C:\Users\Documents\Reports.

6. Space Management
Allocates and deallocates storage space for files efficiently.
Tracks free and used blocks on the disk using structures like free space lists, bitmaps, or linked lists.
Ensures that storage space is reused effectively when files are deleted.
Example: Storing a file without leaving significant gaps of unused space.

7. File Backup and Recovery
Provides mechanisms to create backups of files to prevent data loss.
Implements recovery systems to retrieve data in case of accidental deletion or hardware failure.
Example: Restoring a deleted file from the Recycle Bin or a cloud backup service.

8. File Sharing
Facilitates sharing files among multiple users or applications in a secure manner.
Implements mechanisms like locking to prevent simultaneous conflicting updates.
Example: Sharing a document on a network drive with restricted edit access.

9. File Integrity and Consistency
Ensures that files are not corrupted during storage, retrieval, or transmission.
Implements checks like cyclic redundancy checks (CRC) and hashing.
Ensures proper synchronization in case of concurrent access by multiple processes.
Example: Ensuring that a downloaded file matches its original version.

Q3) What is Spooling?
Spooling (Simultaneous Peripheral Operation On-Line) is a process in which data is temporarily stored in a buffer (usually on disk) to manage input/output (I/O) operations more efficiently. It acts as an intermediary between slow I/O devices and faster CPU processes, allowing multiple processes to share the same I/O resources without being blocked.

Key Features of Spooling
1. **Buffering Mechanism**:
Data is stored in a buffer or queue before being sent to the I/O device.
This decouples the CPU from slower I/O operations.
2. **Parallelism**: Spooling enables parallel execution of processes by overlapping I/O operations with computation.

3. Used for Shared Devices:Commonly used for devices like printers, where multiple jobs need to be processed sequentially.

Detailed Steps in the Working of Spooling

1. Job Submission

When a process generates output or requests input, the data is **stored in a spool** (temporary buffer, typically located on disk).

The spool acts as a queue where multiple jobs can be added for processing.

2. Data Queuing in the Buffer

Jobs are queued in the buffer in the order they are received. This queue ensures **fairness** in handling requests.

Example: In a printer spooler, print jobs are added to the buffer, waiting for the printer to process them.

3. Device Operation

The **I/O device (e.g., printer)** retrieves data from the spool sequentially.

The CPU continues to perform other tasks without waiting for the I/O device to complete

4. Parallel Processing

While the I/O device processes one job, the **CPU processes another job**, adding new data to the spool.

This **overlapping of operations** ensures efficient resource utilization.

5. Job Completion

When the I/O device completes a job, it automatically picks up the next job in the queue.

Completed jobs are removed from the spool.

Working Example: Printer Spooling

Step 1: A user sends three documents (Doc1, Doc2, Doc3) to a shared printer.

Step 2: The spooler queues these documents in the order they are received.

Buffer contents: [Doc1, Doc2, Doc3].

Step 3: The printer retrieves **Doc1** from the spool and starts printing.

Step 4: While the printer is working on **Doc1**, the CPU is free to process other tasks or accept new print requests.

Step 5: Once **Doc1** is printed, **Doc2** is picked up by the printer, and the process continues.

Advantages of Spooling

Efficiency: Overlaps CPU computation and I/O operations, reducing idle time.

Resource Sharing: Allows multiple users or processes to share I/O devices.

Queue Management: Jobs are queued in an orderly manner, ensuring fairness and systematic execution.

Fault Tolerance: If an I/O device fails temporarily, jobs are safely stored in the spool.

```
Step 1: Process Generates Output
+--------+                        +------------+
|  CPU   | -----> Writes ----->|   Buffer   |
+--------+                        +------------+

Step 2: Queue Jobs in Buffer
+------------+
|   Buffer   |
| [Job1]     |
| [Job2]     |
| [Job3]     |
+------------+

Step 3: I/O Device Processes Jobs
+------------+                     +--------------+
|   Buffer   | -----> Reads --->|  I/O Device  |
| [Job2]     |                     | (e.g., Printer) |
| [Job3]     |                     +--------------+
+------------+
```

Q4) What is I/O Buffering and why is it needed? Explain with diagrams different I/O buffering techniques.

The *buffer* is an area in the main memory used to store or hold the data temporarily. In other words, buffer temporarily store data transmitted from one place to another, either between two devices or an application. The act of storing data temporarily in the buffer is called *buffering*.

I/O buffering refers to the temporary storage of data in memory (a buffer) while it is being transferred between devices (such as a disk, keyboard, or network) and the processor. The buffer acts as an intermediary space where data is stored temporarily before being processed or sent to its destination.

Why is I/O Buffering Needed?

1. Speed Mismatch:

I/O devices are typically slower than the CPU. Buffering allows the CPU to perform other tasks while waiting for data, improving efficiency.

2. Smooth Data Transfer:

Buffering helps manage irregular data transfer rates from I/O devices, preventing delays and ensuring continuous data flow.

3.**Handling Variable Data Rates:**

Different I/O devices transfer data at varying speeds. Buffers accommodate these differences, storing data when devices are slow and feeding it to the CPU at a steady rate.

4.**Efficiency in Disk Operations:**

Buffers allow data to be read or written in larger chunks, reducing I/O operation overhead and speeding up file access.

5.**Reduced Latency:** By buffering data, the CPU can process one chunk while the next is being fetched, reducing waiting time.

Types:

1. Single Buffering

A single buffer is allocated in memory to temporarily store data either from an input device or destined for an output device.

Working:

1. **Input Operation**:

The I/O device writes data into the buffer while the CPU processes data from the buffer.

Once the buffer is full, the I/O device pauses until the CPU processes the data.

2. **Output Operation**:

The CPU writes data to the buffer, and the I/O device reads data from the buffer to perform the output operation.

The I/O device must wait for the CPU to fill the buffer.

Advantages:

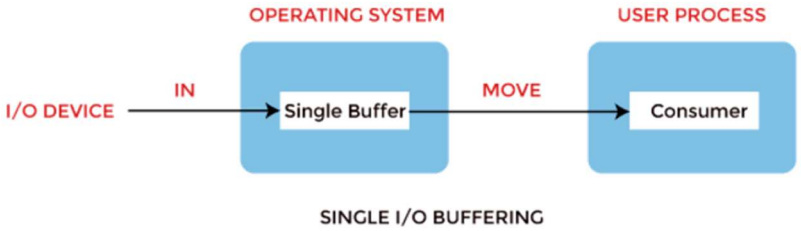
Simple and easy to implement.

Requires minimal memory.

Disadvantages:

Inefficient for large-scale data transfer.

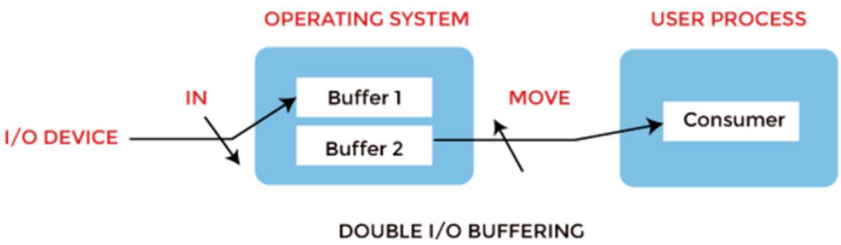
Either the CPU or the I/O device remains idle, leading to suboptimal performance.



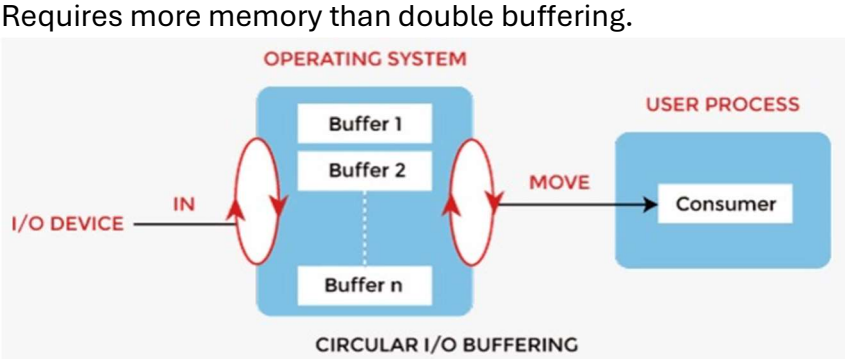
2. Double Buffering

In **Double Buffering**, two schemes or two buffers are used in the place of one. In this buffering, the producer produces one buffer while the consumer consumes another buffer simultaneously. So, the producer not needs to wait for filling the buffer. Double buffering is also known as buffer swapping.

Block oriented: This is how a double buffer works. There are two buffers in the system.
The driver or controller uses one buffer to store data while waiting for it to be taken by a higher hierarchy level.
Another buffer is used to store data from the lower-level module.
A major disadvantage of double buffering is that the complexity of the process gets increased.
If the process performs rapid bursts of I/O, then using double buffering may be deficient.
Stream oriented: It performs these operations, such as:
Line- at a time I/O, the user process does not need to be suspended for input or output unless the process runs ahead of the double buffer.
Byte- at time operations, double buffer offers no advantage over a single buffer of twice the length.
Advantages:
Reduces CPU idle time as processing and I/O operations occur simultaneously.
Suitable for medium to high-speed data transfers.
Disadvantages:
Requires more memory than single buffering.
Slightly more complex implementation than single buffering.



3. Circular Buffering
Multiple buffers are organized in a circular queue, enabling continuous processing of I/O operations.
Working:
The I/O device writes data sequentially into the buffers (Buffer 1 → Buffer 2 → Buffer 3, etc.).
The CPU processes data from these buffers in the same order.
When the last buffer is processed, the system cycles back to the first buffer, maintaining a continuous flow of data.
Advantages:
Efficient for continuous and real-time data streams, such as audio or video processing.
Reduces CPU idle time and eliminates delays caused by I/O speed mismatches.
Disadvantages:
Complex implementation to manage buffer pointers.
Requires more memory than double buffering.



Q5) List and explain different file access methods.
File access methods define how data can be read from or written to a file. These methods are designed to suit various types of applications and data structures. The primary file access methods are:
1. Sequential Access
Data is accessed in a sequential order, one record after the other, starting from the beginning of the file.
This method is simple and efficient for processing all records in the file.

Working:

1. Reading:

The system begins at the start of the file and reads records one by one.

If a specific record is needed, all preceding records must be read first.

2. Writing: New data is appended to the end of the file.

Advantages:

Simple to implement and requires minimal overhead.

Suitable for applications where all records need to be processed, like payroll systems or logs

Disadvantages: Inefficient for accessing specific records since all previous records must be traversed.

Use Case: Processing bank statements, where each transaction is read sequentially.

```
+--------+--------+--------+--------+
| Record1| Record2| Record3| Record4|
+--------+--------+--------+--------+
   ^
   Sequential Access starts from Record1
```

2. Random (Direct) Access

Records can be accessed in any order directly without reading through preceding records.

Each record is assigned a unique logical address, which is used to access the data directly.

Working:

1. Reading:

The logical address of the required record is calculated or retrieved.

The system accesses the specific record using the address.

2. Writing: Data can be modified or replaced directly at the specific location.

Advantages:

Fast access to specific records.

Efficient for applications requiring frequent access to random data points.

Disadvantages:

Requires additional data structures (like indices) or calculations to determine record locations.

Not suitable for sequential processing.

Use Case: Databases, where specific records (e.g., customer details) need to be retrieved quickly.

```
+--------+--------+--------+--------+
| Record1| Record2| Record3| Record4|
+--------+--------+--------+--------+
   Access Record3 Directly
```

3. Indexed Sequential Access

Combines the benefits of both sequential and random access.

An index is maintained that maps logical record numbers to their physical locations in the file.

Data can be accessed either sequentially or directly using the index.

Working:

1. **Reading**:

The system searches the index to locate the address of the desired block or record.

If the record is found in the index, it is accessed directly.

For sequential processing, the index is used to locate the starting point, and records are then read one by one.

2. **Writing**: Data can be added sequentially or inserted at specific positions by updating the index

Advantages:

Provides the flexibility of both sequential and direct access.

Faster than pure sequential access due to the use of the index.

Disadvantages:
Overhead in maintaining the index.
Index rebuilding is needed if records are frequently updated or added.
Use Case: Library catalog systems or large-scale databases.

```
Index Table:
+--------+--------+--------+
| Record | Address| Pointer|
+--------+--------+--------+
    ↓       ↓
+--------+--------+--------+--------+
| Block1 | Block2 | Block3 | Block4 |
+--------+--------+--------+--------+
Access Record by Searching Index
```

Q6) **Typical Operations on a Directory**
1.Create Directory:
This operation involves creating a new directory (folder) in the file system. It helps in organizing files into a hierarchical structure.
Example: Creating a "Documents" directory to store all document files.
2.Delete Directory:
This operation removes an existing directory from the file system. If the directory is not empty, its contents must be deleted first.
Example: Deleting a "Temp" directory after clearing the files within it.
3. Rename Directory:
This operation changes the name of an existing directory.
Example: Renaming a "Work" directory to "Projects".
4. List Directory Contents:
This operation displays a list of all files and subdirectories within a given directory, helping users view and manage the contents.
Example: Listing all files in the "Downloads" directory.
5.Change Directory (cd):
This operation changes the current working directory to another specified directory, allowing the user to navigate the file system.
Example: Using the command cd /home/user/Downloads to navigate to the Downloads directory.
6. Access Control:
Setting permissions or access control for a directory ensures that only authorized users can view, modify, or execute the files inside it.
Example: Granting read/write permissions to a "Shared" directory for specific users.

Q7) **Disk Scheduling Criteria**
Disk scheduling algorithms are used to determine the order in which disk I/O requests (read/write operations) are processed to optimize the efficiency of disk access. The three common criteria for disk scheduling are seek time, rotational latency, and throughput.
1. Seek Time
Seek time is the time it takes for the disk's read/write head (also known as the disk arm) to move from its current position to the track where the data is requested. The seek time is a significant part of the total disk access time.

Types of Seek Time:
Full Seek: The head moves from one end of the disk to the other, which is the maximum distance.
Partial Seek: The head moves within a smaller range of tracks, which reduces the time spent compared to a full seek.
Example: Imagine a disk with 100 tracks, and the current position of the read/write head is track 20. Now, a request arrives to read data from track 70.
The seek time is the time required for the head to move from track 20 to track 70, which is a movement of 50 tracks. Seek time is measured in milliseconds and depends on how fast the disk arm moves.
Disk Scheduling Goal: Minimizing seek time is a major objective in disk scheduling. Algorithms aim to minimize the number of tracks the head moves to reduce the total time spent accessing data.
2. Rotational Latency
Rotational latency is the time it takes for the disk to rotate and position the correct sector under the read/write head after the head has been placed on the correct track. This occurs because data is not stored in a continuous block, but in fixed sectors spread across the disk. The disk must wait for the sector containing the requested data to rotate under the head.
Formula:
Rotational latency can be calculated as the time it takes for the disk to complete a full rotation, divided by the number of sectors per rotation, then multiplied by the number of sectors the head must wait for.
Example:
Let's say the disk rotates at 7200 RPM (revolutions per minute). The time for one full rotation would be:

$$\text{Rotation time} = \frac{60}{7200} = 8.33\,\text{ms per rotation}$$

If the read/write head is in the worst case waiting for the disk to complete half a rotation, the rotational latency would be approximately 4.17 ms.
Disk Scheduling Goal: Minimizing rotational latency is essential to improve disk I/O efficiency. Disk scheduling algorithms attempt to reduce waiting times caused by rotational delays.
3. Throughput
Throughput refers to the rate at which I/O operations are completed, often expressed as the number of operations (such as read/write requests) that can be completed in a unit of time (e.g., operations per second). High throughput is desirable because it means that the disk is efficiently handling many requests over a short period.
Key Factors Affecting Throughput:
1. **Disk Access Time**: The overall time taken to process a request, including both seek time and rotational latency.
2. **Queue Management:** The number of I/O requests waiting to be processed; a longer queue can lower throughput.
3. **Disk Utilization**: The extent to which the disk is actively being used. High disk utilization generally results in better throughput.
Example: If a disk handles 100 read/write operations per second, the throughput of the disk is 100 operations per second.
For a disk with 1000 operations and an average seek time of 10 milliseconds and rotational latency of 4 milliseconds, the throughput can be impacted by how efficiently the system schedules the disk requests to minimize these delays.
Disk Scheduling Goal: Maximizing throughput is achieved by reducing delays and optimizing the order in which requests are processed. This ensures that more operations can be completed in a given period.
3. Transfer Time
Transfer time is the time required to transfer data between the disk and the system once the read/write head has positioned itself over the correct track and the correct sector is aligned.
This is influenced by factors like the disk's data transfer rate (measured in MB/s or KB/s). The transfer rate is determined by the technology used in the disk (e.g., hard drive or SSD).
Formula for Transfer Time:

$$\text{Transfer Time} = \frac{\text{Size of Data}}{\text{Transfer Rate}}$$

Example:
Assume a disk has a transfer rate of 10 MB/s, and you need to transfer a 100 KB file.

$$\text{Transfer Time} = \frac{100\ \text{KB}}{10\ \text{MB/s}} = \frac{100}{10240} = 0.0098\ \text{seconds} = 9.8\ \text{ms}$$

In this case, the transfer time is the time it takes for the actual data to be read from or written to the disk after the head has found the correct sector.

Q8)Disk Scheduling Algorithms
Disk scheduling algorithms determine the order in which disk I/O requests (seeking a particular disk block) are processed.

1. First-Come, First-Served (FCFS) Scheduling

FCFS is the simplest disk scheduling algorithm. It processes requests in the exact order they arrive without any reordering.
The **disk arm** moves from the current position to the requested position for each I/O request in the order of their arrival.

Working:
Requests are processed in the order they arrive.
The disk arm moves directly to the requested track for each request.
Example: Given disk requests: **40, 10, 20, 30**, starting at **50**.
The disk arm starts at **50** and first goes to **40**, then **10**, then **20**, and finally **30**.
Seek sequence: 50 → 40 → 10 → 20 → 30.
Disadvantages:
Inefficient: Can lead to very high **seek time** as requests might be far apart.
Non-optimal for high throughput.

2. Shortest Seek Time First (SSTF) Scheduling:

SSTF selects the request that is closest to the current position of the disk arm, minimizing the seek time.
This approach tends to improve **seek time**, but it can lead to **starvation** (requests far from the current head may never be processed).

Working:
From the current position, the algorithm selects the request that requires the shortest distance (seek time).
This continues until all requests are completed.
Example: Given disk requests: **40, 10, 20, 30**, starting at **50**.
The disk arm starts at **50**.
First, it moves to **40** (closest to 50).
Then, from **40**, it moves to **30** (closest to 40).
Then, from **30**, it moves to **20** (closest to 30).
Finally, from **20**, it moves to **10**.
Seek sequence: 50 → 40 → 30 → 20 → 10.
Disadvantages:
Starvation: Requests far from the current position may be delayed indefinitely.
May not always give the most **optimal** solution for throughput.

3. SCAN Scheduling (Elevator Algorithm)

SCAN moves the disk arm in one direction, servicing requests until it reaches the last track, then reverses direction and processes the requests in the opposite direction.
It's called the **elevator algorithm** because it resembles the movement of an elevator, going to the top or bottom before reversing direction
Working: The disk arm starts at one end of the disk and moves towards the opposite end, servicing requests as it goes.

After reaching the last track, it reverses direction and services requests in the opposite direction.
Example:
Given disk requests: **40, 10, 20, 30**, starting at **50**, with disk limits from **0 to 100**.
The disk arm starts at **50** and moves towards the higher-numbered tracks (increasing direction).
It will first service **40**, **50**, then go to **100** (the end), then reverse direction and service **30**, **20**, and **10**.
Seek sequence: 50 → 40 → 30 → 20 → 10 → 100 → 90.
Advantages:
Reduces travel distance compared to FCFS.
Can be efficient for systems with requests spread out across the disk
Disadvantages:
Can result in **long wait times** for requests that are just behind the disk arm as it reverses direction.
Starvation: Requests that are in the opposite direction to the head's current motion can be delayed.

4. LOOK Scheduling
LOOK is similar to **SCAN**, but instead of going to the end of the disk, the disk arm only moves as far as the last request in the current direction.
It stops as soon as it has serviced all the requests in the direction of travel and then reverses direction.
Working:
The disk arm moves in one direction, servicing all requests, but only goes as far as the last request before reversing direction.
Example: Given disk requests: **40, 10, 20, 30**, starting at **50**, with disk limits from **0 to 100**.
The disk arm starts at **50**, moving towards the higher-numbered tracks (increasing direction).
It services **40** and **30** before reversing direction.
Then it services **20** and **10**.
Seek sequence: 50 → 40 → 30 → 20 → 10.
Advantages:
Reduces unnecessary movement (no need to go to the absolute end of the disk like in SCAN).
More efficient than SCAN when there are no requests at the extreme ends
Disadvantages: Starvation: Requests in the opposite direction can still experience delays.

5. C-SCAN (Circular SCAN) Scheduling
C-SCAN is a variation of SCAN, where the disk arm moves in one direction to the end of the disk and then **returns to the beginning** without servicing any requests during the return.
After reaching the end, it **jumps back** to the beginning of the disk and continues servicing requests in the same direction.
Working:
The disk arm moves towards the end of the disk, servicing requests
After reaching the last track, it immediately jumps to the beginning of the disk and starts servicing requests again in the same direction.
Example: Given disk requests: **40, 10, 20, 30**, starting at **50**, with disk limits from **0 to 100**.
The disk arm starts at **50** and moves towards the higher-numbered tracks.
It services **40**, then reaches the end of the disk (100).
Then it **jumps back** to **0** and services **10**, **20**, and **30**.
Seek sequence: 50 → 40 → 100 → 0 → 10 → 20 → 30.
Advantages:
Prevents starvation because requests are serviced in a circular manner.
More **uniform response time** compared to SCAN.
Disadvantages:
Long jump time: The arm jumps to the beginning, which may increase overall latency.
Requests near the middle may experience delays.